University of Toronto

Faculty of Applied Science & Engineering

ECE496 Implementation Plan

Project Title	Building an Intelligent Job Scheduler for Optimized Request Assignment
Project Number	PROJ 521
Supervisor	Ali Bereyhi
Administrator	Sarah Oliveira
Prepared By	Housen Zhu Ruoyu Li Yifan Wang Yuan Sui
Date of submission	11/15/2024

1. Project Status and Changes

Our team has been divided into frontend and backend groups to efficiently manage the development of the intelligent job scheduler. Progress in both domains has been steady, with foundational components already implemented on time.

1.1 Current Status of Project Components

At present, the backend team has successfully completed the majority of the project environment setup. The environment is written in Python, and the main schedule functionality is completed. This environment has been uploaded to GitHub, allowing for version control and streamlined collaboration [1].

Our team is steadily moving forward with foundational learning Deep Reinforcement Learning. We are utilizing materials provided by our professors, including lecture videos, notes, and Q&A sessions with our professors bi-monthly, to prepare for the next step in the project's model development.

The data structure has been established and is divided into two main categories—dynamic data and environment data. Dynamic data can now be randomly generated as required, and samples of environmental data have been manually created to provide the necessary datasets for initial testing.

In parallel, the frontend team has developed the main interface for the website. In alignment with our original web prototype, we have implemented several minor optimizations to enhance the overall usability and visual appeal of the website. Other than that, the chatbot has been implemented with the ability to handle simple input and output interactions, which is already deployable on a shareable website.

1.2 Challenges and Obstacles

As expected, there have been no major changes to the project since the submission of the Final Proposal, but we have encountered some obstacles. During the environment setup process, we encountered a problem where the output generated according to the resources we found did not provide clear visibility into the source of potential issues. Since the output lacks clarity, it is difficult to determine where the errors are occurring, which prompted the need to rewrite the output format. Additionally, all team members are relatively inexperienced with front-end development, leading to some common mistakes throughout the process.

2. Possible Solutions and Design Alternatives

At this stage, we evaluated one primary design alternative at the system level for our intelligent job scheduler, which is the weighted rule-based system. This approach draws on traditional scheduling methods, such as prioritizing requests based on their severity or arrival time, while incorporating statistical weighting for finer-grained prioritization [2]. This type of scheduler, commonly applied in queue and lineup management systems, is straightforward in design and relatively easy to implement. It offers a high level of predictability and stability, as it follows a fixed set of weighted rules, making it ideal for handling standard cases, where requirements are predictable.

Consider a hospital emergency department using a weighted rule-based scheduling system to manage patient requests. For instance, a patient experiencing chest pain might receive a higher weight due to the potential urgency of their condition, while a patient with a minor injury, like a sprained ankle, would receive a lower weight. The system would process each case by assigning a weighted score based on these factors. In summary, patients with higher scores indicating more severe conditions or time constraints will be prioritized over those with lower scores. This structure ensures that, under normal demand, the system handles patients in a way that aligns with traditional triage rules.

However, the weighted rule-based system has notable limitations when applied to the dynamic and unpredictable environment of an emergency department. It operates solely on predefined rules without learning from past scheduling results. Moreover, it cannot optimize resource allocation in real time or adapt to fluctuations in demand, which is crucial for emergency applications where conditions can change rapidly from time to time.

3. Technical Design and Implementation

On the system-level, this project consists of an application frontend and backend subsystems. First, a web-based Graphical User Interface (GUI) frontend that will interact with patients, doctors and hospital administrators. Second, a Deep-Q-Network (DQN) based backend scheduling algorithm that will manage patient status and medical resources to minimize patient wait time. The team is divided into frontend and backend to develop the GUI and scheduling algorithm respectively.

3.1 System-Level Overview

The front end of this project acts as the main layer for user interaction, allowing access to the intelligent job scheduler through the website. It’s designed with both simplicity and functionality in mind and is built around four main components: the main page, login page, scheduler, and chatbot interface.

The main page provides basic information and includes links to an About/Contact Page and the Login Page. Since the scheduler is used by both patients and doctors, the login function differentiates between these two user groups. Once logged in, users are directed to the features relevant to their roles. Doctors primarily use the Patient Input module, where they can enter key patient details, such as "Patient Name" and "Acuity Level." This data is then sent to the backend, where the scheduler processes it before returning the results to the doctor.

Patients, on the other hand, primarily interact with the Chatbot module. The chatbot offers real-time updates and responds to questions, providing information on wait times, resource availability, and other updates based on the scheduler's processing of patient input. This feature improves patient experience and helps them understand their position in the queue and when they can expect treatment.

The implementation of the backend algorithm consists of three phases: design, training and integration. In the initial design phase, the team will implement the backend system in python which consists of an emergency department (ED) simulator module and Deep Reinforcement Learning (DRL) algorithm module. The ED simulator keeps track of patients waiting for each medical resource and the state of occupancy for each resource. The DRL algorithm selects the next patient to be attended by the medical resource and updates the ED simulator of the decision. The ED simulator will then manage the new state of occupancy for each medical resource according to the DRL algorithm decision.

In the second training phase, the team will generate realistic patient data (e.g. arrival time and treatment pattern) that is statistically similar to what is observed in hospital emergency departments. The generated data will be used to train the algorithm to learn how to best make decisions to minimize patient wait time. Each time the algorithm makes a patient selection, the decision and new state of medical resources is stored in the memory module. After a number of decisions are made, the algorithm learns from past memory and updates DRL algorithm weights. With hours of generated data training, the Q-network will be optimized to make the selection that minimizes patient waiting time in the hospital.

In the final integration phase, the team will connect the backend scheduling algorithm to the frontend GUI for ease of use in the hospital setting. As a patient arrives, their information will be logged in to the system and the algorithm will automate the scheduling of medical resources. The training of algorithms continues during deployment and will become even more optimized as the network learns and adapts to the specific hospital environment.

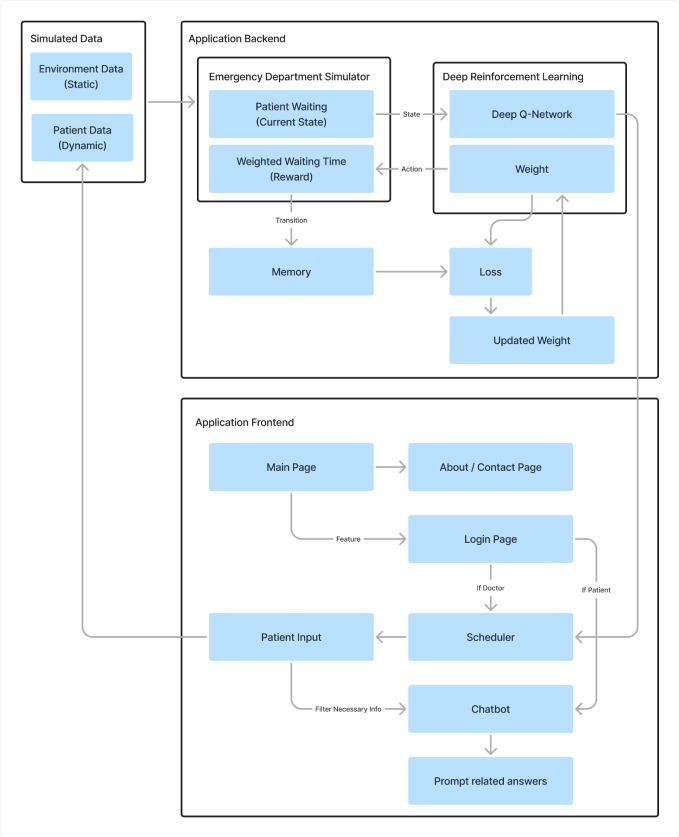


Figure 1: System block diagram [3]

3.2 Module-Level Descriptions

The **Simulated Data module** consists of pre-configured **environment data** and randomly-generated **patient data**. The environment data is specific to each hospital setting. In our design, we model the hospital to have four types of medical resource with a set number of personnels for each type: three Doctors, five Nurses, one X-ray and one CT technician. The patient data is randomly generated according to statistically realistic distribution . For example, the patient arrival time is generated by a poisson random process, the duration of patient treatment is modeled by gaussian and exponential distribution. Each patient data consists of arrival time, acuity level (a number between 1-5 that measure level of disease severity), treatment plan (a sequence of treatment that are to be attended by medical resources) and duration of each treatment.

Within the **Application Backend** subsystem, the **Emergency Department Simulator module** is responsible for logging the patient information in the resource waiting line as they come and updating the medical resource waiting line as they finish treatment. The distribution of patients waiting for medical resources is called the **state**. The **Deep Reinforcement Learning module** is a three layer neural network that takes in patient information and the state of each medical resource and decides which patient in the waiting line should receive the treatment next, which is called an **action**. The action of the Q-network is non-deterministically chosen by the **weight** of the network, which is trained and updated by the **loss**

function to minimize patient wait time. After each action is completed, a weighted patient waiting time is calculated and considered as a **reward**. Then this **transition,** which consists of action, reward, previous and updated state, are stored in the memory for later use in training. Ultimately, the Q-network will be optimized to schedule medical resources in the most time-efficient way.

In addition, the **Application Frontend** subsystem is designed with a main page that provides access to features like the **About/Contact** and **Login** pages. Users logging in are directed to different workflows based on their roles. Doctors are directed to a **scheduler**, while patients are routed to a **Patient Input** interface that collects necessary information. It can facilitate the scheduling process. A **Chatbot** is integrated to provide prompt-related answers. The front end is structured to optimize the using experience for both patients and healthcare providers.

4. Design Requirements and Verification Table

In the previous proposal, we specified the project requirements of the intelligent job scheduler (Appendix A). Now we have adjusted the requirements to make it align well with our current scope, and we combine the verification methods into the verification table to evaluate the project's success. Details will be demonstrated in the table below.

Verification Table:

ID	Project Requirement	Verification Method
1	Weighted average delay experienced by users: lower than the baseline scheduler delay	Test 1: test the weighted average delay of the job scheduler using the baseline model as the benchmark Test 2: test the weighted average delay of our final job scheduler using DQN Success criteria: the final job scheduler should have less delay than the baseline scheduler
2	Chatbot: proper interaction	Test: ask the chatbot for query time to check whether the chatbot can respond properly Success criteria: the chatbot should respond with the same query information as the backend output
3	GUI complexity: no more	Test: check the number of clicks needed to find the login button

	than 3 clicks [4] for main functionalities	before logging in, and check the number of clicks needed to find the logout button and the chatbot after logging in. Success criteria: it should take no more than 3 clicks to find the login button before logging in, and the logout button and the chatbot after logging in [4].
4	User interface response time: maximum 1 second [4]	Test: click the login button and the logout button and count for the response time. Success criteria: the response time should be less than 1 second [4]
5	User information: must be saved after logging out	Test: log in as a new user then log out, then log in again to see whether the account information is saved. Success criteria: the account information should be saved and displayed when the user logs in again
6	User Interface: the login and logout functions should perform properly	Test 1: click the login button and type the wrong username to check whether there is an error message. Success criteria 1: there should be error message Test 2:click the login button and type the wrong password to check whether there is an error message Success criteria 2: there should be error message Test 3: click the refresh button on the browser after logging in Success criteria 3: the user should remain logged in Test 4: log in as a doctor and run the simulator then log in Success criteria 4: the functions in the doctor interface should not be accessible for the anonymous user

		Test 5: log in as a patient and ask the chatbot with “show me the query” then log out Success criteria 5: the functions such as chatbot in the patient interface should not be accessible for the anonymous user
7	User interface: each button should function properly	Test: click each redirection button and inspect whether they can redirect to the correct page Success criteria: all the redirection button should redirect to the correct page
8	User interface: should properly handle wrong inputs	Test: type 6, -1, and 4.5 in acuity level to see whether there is an error message for the out-of-bound acuity level Success criteria: there should be error message for error inputs

5. Work Plan

As a group of four computer engineering students, each member has a specialized software-focused skill set for the project. Yifan Wang is responsible for front-end design, developing user-friendly GUI and chatbot interface. Housen Zhu and Yuan Sui focus on backend tasks, with one dedicated to designing and another focused on testing. The Ruoyu Li bridges both front-end and backend design and supports seamless functionality.

5.1 Work Breakdown Structure

Task #	Task	Housen	Ruoyu	Yifan	Yuan
● Preliminary Research					
1	● In-Depth Background Study	A	A	R	A
2	● System Architecture Design	A	A	A	R
● Data					

3	• Data Acquisition	A			R
4	• Data Preprocessing	R	A	A	A
• Prototype Development					
5	• Markov Decision Process Simulation	T	A	A	R
6	• Chatbot Interface Design		A	R	
7	• Main Website Design		R	A	
8	• DRL Model	R	T		A
9	• Chatbot Interface Development		A	R	
10	• Main Website Development		R	A	
• Comprehensive Testing					
11	• System Integration	A		R	
12	• Functional Unit Test		A		R
• System Optimization					
13	• Model Fine-Tuning	R			A
14	• Login Page		R	A	
• Final Deployment					
15	• Final Testing and Validation	A	R	A	A

R = Responsible, A = Assisting, T= Testing

Task 1 & 2 are done by the lead of Yifan and all other members before the start of the semester. We researched the possible solutions on existing scheduling algorithms and DRL applications for the problem statement.

Task 3: Generate the simulated data for effective training of the DRL model.

Task 4: Finalize the dataset and prepare for model training (training and validation sets).

Task 5: Implement the prototype code for DQN - Markov Decision Process Simulation.

Task 6 & 7: Design two front end interfaces.

Task 8: Implement the deep reinforcement learning code - DQN.

Task 9 & 10: Develop two front end interfaces and connect with the back end.

Task 11: Integrate all individual system modules.

Task 12: Design and conduct the unit tests for all system modules.

Task 13: Fine tuning the DQN model - Evaluation, Refinement and Validation.

Task 14: Implement the login page.

Task 15: Deploy the scheduler in a controlled environment for final testing and validation.

5.2 Gantt Chart

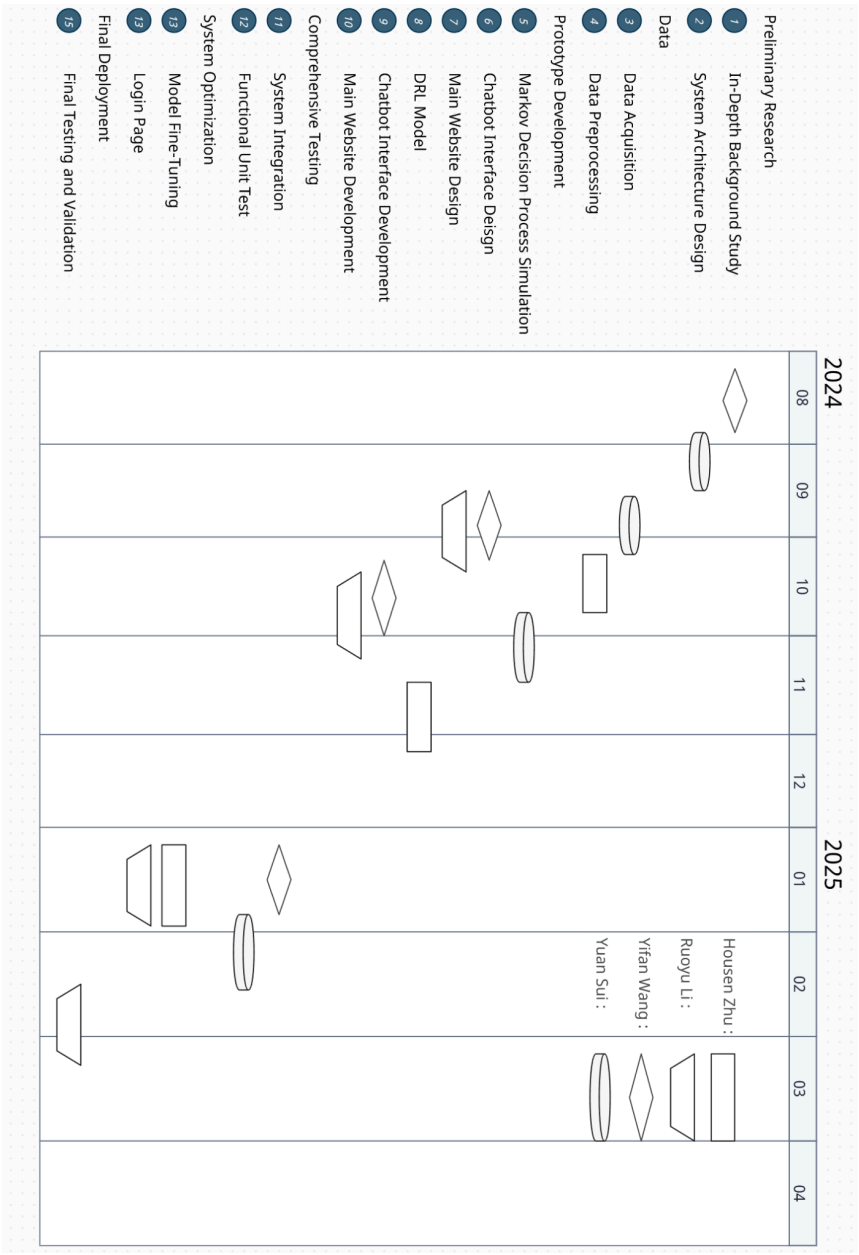


Figure 2: Gantt Chart

5.3 Financial Plan

Consumables & Services

Item	Priority	Cost/unit	Quantity (# of hours)	Total Cost	Requires Funding
Website	1	$52	3 years	$154	y
Colab membership	1	$14	4*4 months	$224	y
Total Consumables / Service				$378	
Total Requiring Funding				$378	
			Funding		
			Students ($100 ea)		$378
			Total Funding		$378

Student Labour

Item	Cost/unit	Quantity (# of hours)	Total Cost
Housen Zhu	$7	600	$4200
Ruoyu Li	$7	600	$4200
Yifan Wang	$7	600	$4200
Yuan Sui	$7	600	$4200
Total Student Labour (unfunded)			$16800
Total Cost of Project			$17178
Total Cost Requiring Funding			$378

Figure 3: Budget table [5]

Since our product is all on the software end, we do not need fundings for any hardware or equipment. If the final fundings could not afford the fees of Website or Colab membership, we would instead borrow the website for free and train our model in local devices.

6. Reference

[1] Yifan Wang, Building an Intelligent Job Scheduler for Optimized Request Assignment. GitHub repository. Accessed: Nov. 14, 2024. [Online]. Available: https://github.com/wang2653/Building-an-Intelligent-Job-Scheduler-for-Optimized-Request-Assignment

[2] Statistics Canada. How to Cite Statistics Canada Products. Accessed: Nov. 14, 2024. [Online]. Available: https://www150.statcan.gc.ca/n1/edu/power-pouvoir/ch6/5214809-eng.htm

[3] Yifan Wang, PROJ 521 Board. Figma. Accessed: Nov. 14, 2024. [Online]. Available: https://www.figma.com/board/7w1LAtLJW4rs0QIWcY6Xzw/PROJ-521?node-id=1-359&t=Ie4JIciXhouFXgwb-1

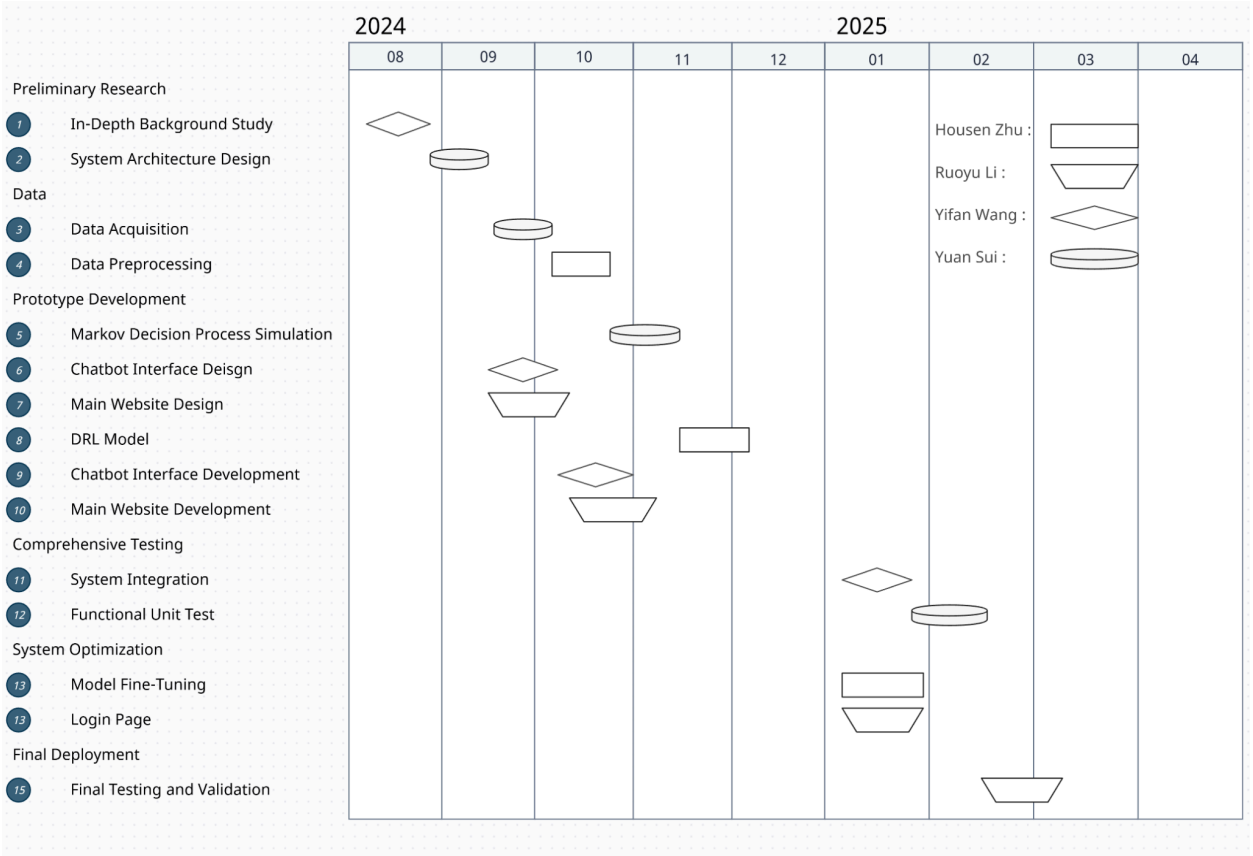
[4] Simform. *Building Scalable Applications on AWS: Best Practices & Architecture*. Accessed: Nov. 14, 2024. [Online]. Available: https://www.simform.com/blog/building-scalable-application-aws-platform/

[5] “Com domains: Buy your domain name - godaddy CA,” GoDaddy, https://www.godaddy.com/en-ca/tlds/com-domain (accessed Nov. 14, 2024).

7. Appendices

Appendix A: Project Requirement Table

ID	Project Requirement	Description
1	Allocation of incoming requests to agents.	Functional: the scheduler must allocate the incoming patient requests to medical resources.
1A	DRL-based agent assignment	Constraint: The system must assign incoming requests using a DRL algorithm.
1B	Minimize the average delay experienced by users	Objective: The average delay using DRL should be lower than that of the rule-based algorithm.
1C	Concurrent request handling	Objective: the system should handle at least 1000 concurrent user requests .
2	Chatbot for user interaction	Functional: the chatbot should be able to show treatment schedules and handle initial interactions with users.
3	Network load prediction	Functional: th·the system must predict incoming request load, including volume and distribution.
4	User-friendly GUI	Functional: the system must have a GUI.
4A	Minimize the GUI complexity	Objective: the GUI should allow users to perform key tasks in no more than 3 clicks .
4B	User interface response time	Objective: the system should operate with a response time of less than 1s per action in UI.
5	System development cost	Constraint: the cost of developing the system must not exceed 400 CAD.
6	Data privacy compliance	Constraint: the system must comply with all relevant legal and privacy regulations when handling user data.


2024
2025
08
09
10
11
12
01
02
03
04
Preliminary Research
1 In-Depth Background Study
2 System Architecture Design
Data
3 Data Acquisition
4 Data Preprocessing
Prototype Development
5 Markov Decision Process Simulation
6 Chatbot Interface Deisgn
7 Main Website Design
8 DRL Model
9 Chatbot Interface Development
10 Main Website Development
Comprehensive Testing
11 System Integration
12 Functional Unit Test
System Optimization
13 Model Fine-Tuning
13 Login Page
Final Deployment
15 Final Testing and Validation
Housen Zhu :
Ruoyu Li :
Yifan Wang :
Yuan Sui :